Lab MA: Multi-Agent Systems

with LangChain & LangGraph

IST 488 — Building Human-Centered AI Applications

Time: ∼40 minutes

Overview

In this lab, you will build a **Multi-Agent Trip Planner** as a Streamlit app. A **Supervisor** agent will orchestrate three specialist agents — *Research*, *Budget*, and *Itinerary* — to collaboratively plan a trip. No single agent can produce a complete plan; they must coordinate through the Supervisor.

By the end of this lab you will:

- Define custom tools using LangChain's `@tool` decorator
- Create specialist agents with `create_react_agent()`
- Wire them together under a Supervisor using `create_supervisor()`
- Build a Streamlit UI to interact with the multi-agent system
- Compare multi-agent output against a single-agent baseline
- Build a chatbot using `MessagesState` and `StateGraph`

Setup

1. Install the required packages:

```
pip install streamlit langchain langchain-openai langgraph
    langgraph-supervisor
```

2. Make sure your OpenAI API key is configured in `.streamlit/secrets.toml`:

```
OPENAI_API_KEY = "sk-..."
```

3. You are provided a file called `travel_data.json` which contains all the destination information, cost data, and activity pools. Place this file in your `LABS-IST488` directory.
4. Create a new file called `Lab_MA.py` in your `LABS-IST488` directory.
5. Add `Lab_MA.py` to your `streamlit_app.py` navigation so you can access it.

1 Part 1: Setup & Environment (~5 minutes)

At the top of your `Lab_MA.py` file, add the following imports:

```
import os
import json
import streamlit as st
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langgraph.prebuilt import create_react_agent
from langgraph_supervisor import create_supervisor
```

Then set up the page:

- Add a title: "Lab MA: Multi-Agent Trip Planner"
- Add a brief description explaining how the system works (a Supervisor coordinates three specialist agents: Research, Budget, Itinerary)
- Load the OpenAI API key from `st.secrets` (same pattern as earlier labs)
- Create **two** `ChatOpenAI` instances:

```
agent_llm      = ChatOpenAI(model="gpt-4o-mini", temperature=0)
supervisor_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

When complete, run `streamlit run streamlit_app.py` and verify your page loads with the title and description.

2 Part 2: Building Specialist Agents (~10 minutes)

In this section you will define three tools and three agents.

Step 2A: Load the Travel Data

Load the `travel_data.json` file at the top of your tools section:

```
with open("travel_data.json", "r") as f:
    TRAVEL_DATA = json.load(f)
```

The JSON file has this structure:

- `destinations`: dict of city names → {highlights, best_time, culture, tips, weather}
- `flight_estimates`: dict of city names → estimated roundtrip flight cost (USD)
- `daily_costs`: dict of budget levels → {accommodation, food, transport, activities, misc}
- `activities`: dict of interest categories → list of activity strings
- `money_saving_tips`: list of tip strings

Step 2B: Define the Tools

Each tool is a Python function decorated with `@tool`. The decorator automatically registers the function's name, docstring, and parameter types so that an LLM agent can call it:

```
@tool
def my_tool_name(param1: str, param2: int) -> str:
```

```
    """Description of what this tool does. The LLM reads this."""
    # ... your logic ...
    return json.dumps(result)
```

Create **three** tools that pull data from `TRAVEL_DATA`:

1. `search_destination(query: str) -> str`
   Purpose: Look up travel info about a destination.
   Implementation: Read `TRAVEL_DATA["destinations"]`. Match the user's query against city names (case-insensitive). Return the matching city's data as a JSON string. If no city matches, return generic fallback info.

2. `calculate_budget(destination: str, days: int, budget_level: str) -> str`
   Purpose: Estimate trip costs.
   Implementation: Read `TRAVEL_DATA["daily_costs"]` and `TRAVEL_DATA["flight_estimates"]`. Look up the daily rate for the given budget level, calculate totals based on the number of days, add the flight cost. Include `TRAVEL_DATA["money_saving_tips"]`. Return as JSON.

3. `create_schedule(destination: str, days: int, interests: str) -> str`
   Purpose: Build a day-by-day itinerary.
   Implementation: Read `TRAVEL_DATA["activities"]`. Parse the interests string, collect matching activities from the pool, and build a schedule with morning/afternoon/evening slots for each day. Return as JSON.

TIP

These tools return **simulated data** from the JSON file — you are not calling real APIs. This keeps the lab self-contained and predictable. The LLM still reasons about the data and composes a natural-language response.

Step 2C: Create the Agents

Use `create_react_agent()` to create three specialist agents:

```
agent = create_react_agent(
    model=agent_llm,
    tools=[your_tool],
    name="agent_name",
    prompt="You are a specialist in ... Always use the tool ..."
)
```

Create these three agents:

1. **research_agent** — Tools: `[search_destination]`
   Prompt: Tell it to be a travel research specialist that always uses the `search_destination` tool.

2. **budget_agent** — Tools: `[calculate_budget]`
   Prompt: Tell it to be a budget specialist that always uses the `calculate_budget` tool.

3. **itinerary_agent** — Tools: `[create_schedule]`
   Prompt: Tell it to be an itinerary specialist that always uses the `create_schedule` tool.

IMPORTANT

In each agent's prompt, instruct it to **ALWAYS** call its tool and not make up information. This ensures the agent uses the tool rather than relying on its own knowledge.

3 Part 3: Creating the Supervisor (~8 minutes)

The Supervisor is the central orchestrator. It receives the user's request, decides which agent(s) to delegate to, and synthesizes the final response.

```
workflow = create_supervisor(
    agents=[research_agent, budget_agent, itinerary_agent],
    model=supervisor_llm,
    prompt="..."
)
```

Write a supervisor prompt that:

- Lists the three available agents and what each one does
- Instructs the supervisor to route questions:
  - Destination questions → `research_agent`
  - Cost/budget questions → `budget_agent`
  - Schedule/itinerary questions → `itinerary_agent`
  - Full trip planning → **ALL** three agents
- Tells the supervisor to synthesize all responses into one organized plan

Then compile and invoke:

```
multi_agent_app = workflow.compile()

result = multi_agent_app.invoke({
    "messages": [{"role": "user", "content": "your query here"}]
})

# The final response:
result["messages"][-1].content
```

4 Part 4: Streamlit Interface (~10 minutes)

Build the user interface to interact with your multi-agent system.

Step 4A: Sidebar Controls

In a `with st.sidebar:` block, create:

- A `text_input` for the destination (default: "Paris")
- A `slider` for trip duration (1–14 days, default: 5)
- A `selectbox` for budget level (Budget / Moderate / Luxury)
- A `multiselect` for interests (Food, History, Art, Nature, etc.)

Step 4B: Session State

Initialize session state variables to persist results across reruns:

```
if "ma_result" not in st.session_state:
    st.session_state.ma_result = None
if "ma_messages" not in st.session_state:
    st.session_state.ma_messages = None
```

Step 4C: Query Construction & Invocation

Build a query string from the sidebar inputs, e.g.:

> *"Plan a 5-day trip to Paris. My budget level is moderate. My interests include: Food, History. Please provide destination research, a budget breakdown, and a day-by-day itinerary."*

Create a "Plan My Trip" button. When clicked: show a spinner, invoke `multi_agent_app`, store the result in session state, and display it with `st.markdown()`.

Step 4D: Agent Activity Log

After displaying results, add a debug section in the sidebar showing which agents were called and in what order:

```
for msg in result["messages"]:
    msg_name = getattr(msg, "name", None)       # agent name
    tool_calls = getattr(msg, "tool_calls", None) # tool invocations
```

Display the execution order with emoji labels (e.g., research_agent, budget_agent, itinerary_agent).

5 Part 5: Single-Agent Comparison (~7 minutes)

Add a section below the trip plan that compares multi-agent output with a single-agent response.

1. Add a divider and subheader: "Experiment: Single Agent vs Multi-Agent"
2. Add a button "Run Single-Agent Comparison". When clicked, send the **exact same query** to a single LLM with no tools:

```
single_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
single_result = single_llm.invoke(trip_query)
st.markdown(single_result.content)
```

3. Compare the two responses side-by-side. Note the differences in data quality, structure, and specificity. You will reflect on these observations in the Write-Up Questions at the end of this document.

6 Part 6: Chatbot Mode — `MessagesState` & `StateGraph` (~10 min bonus)

In Parts 1–5 you used `create_supervisor()`, a **high-level** helper. Now you will build a conversational chatbot using the **lower-level** LangGraph primitives.

Key Concepts

`MessagesState`

A `TypedDict` that holds a list of messages as the graph's shared state. Every node reads from and writes to this state. Think of it as the "conversation memory" flowing through the graph.

```
from langgraph.graph import MessagesState
# Equivalent to:
# class MessagesState(TypedDict):
#     messages: list[BaseMessage]
```

`StateGraph`

The core graph builder. You define **nodes** (processing functions) and **edges** (connections), then compile into a runnable app.

```
from langgraph.graph import StateGraph, START, END
```

Nodes

Python functions that take the current state and return updated messages:

```
def my_node(state: MessagesState):
    messages = state["messages"]      # read
    # ... process ...
    return {"messages": [response]}  # write
```

Edges

Three types of connections:

- **Static:** `graph.add_edge("node_a", "node_b")`
- **Conditional:** `graph.add_conditional_edges("node_a", router_fn, mapping)`
- **START/END:** `graph.add_edge(START, "first_node")`

Step 6A: Define Graph Nodes

Create these node functions:

1. **supervisor_node** — reads messages, uses the LLM with a routing prompt to output one word: "research", "budget", "itinerary", "all", or "chat"
2. **research_node** — invokes the `research_agent` from Part 2
3. **budget_node** — invokes the `budget_agent`
4. **itinerary_node** — invokes the `itinerary_agent`
5. **run_all_agents** — sequentially invokes all three agents, combines outputs
6. **synthesizer_node** — produces a final, well-organized response from all agent outputs

Step 6B: Define the Router

```
def route_from_supervisor(state: MessagesState):
    last_msg = state["messages"][-1].content.strip().lower()
    if "research" in last_msg:
        return "research"
    elif "budget" in last_msg:
        return "budget"
    elif "itinerary" in last_msg:
        return "itinerary"
    elif "all" in last_msg:
        return "all"
    else:
        return "chat"
```

Step 6C: Build the StateGraph

```
chatbot_graph = StateGraph(MessagesState)

# Add nodes
chatbot_graph.add_node("supervisor", supervisor_node)
chatbot_graph.add_node("research", research_node)
chatbot_graph.add_node("budget", budget_node)
chatbot_graph.add_node("itinerary", itinerary_node)
chatbot_graph.add_node("all_agents", run_all_agents)
chatbot_graph.add_node("synthesizer", synthesizer_node)

# Edges
chatbot_graph.add_edge(START, "supervisor")

chatbot_graph.add_conditional_edges(
    "supervisor",
    route_from_supervisor,
    {
```

```
        "research": "research",
        "budget": "budget",
        "itinerary": "itinerary",
        "all": "all_agents",
        "chat": "synthesizer",
    },
)

chatbot_graph.add_edge("research", "synthesizer")
chatbot_graph.add_edge("budget", "synthesizer")
chatbot_graph.add_edge("itinerary", "synthesizer")
chatbot_graph.add_edge("all_agents", "synthesizer")
chatbot_graph.add_edge("synthesizer", END)

chatbot_app = chatbot_graph.compile()
```

The resulting graph:

```
START
  |
supervisor
  |
  +---> research ----+
  +---> budget ------+---> synthesizer ---> END
  +---> itinerary ---+
  +---> all_agents --+
  +---> synthesizer -+
```

Step 6D: Streamlit Chat Interface

Build a chat interface using `st.chat_input` and `st.chat_message`:

1. Initialize session state for chat history
2. Display existing chat history (loop through messages)
3. On new user input: append to state, invoke `chatbot_app`, extract and display response
4. Try these flows:
   - "Tell me about Tokyo" (routes to research)
   - "How much would that cost for 5 days?" (routes to budget)
   - "Plan me a full trip to Paris" (routes to all agents)
   - "Thanks!" (routes to chat/general)

Testing Your App

Run your completed app: `streamlit run streamlit_app.py`

Verify the following:

☐ The page loads with the title and description
☐ Sidebar controls work (destination, days, budget, interests)
☐ Clicking "Plan My Trip" produces a comprehensive trip plan
☐ The Agent Activity Log shows all three agents were called
☐ The single-agent comparison produces a noticeably different response

- ☐ Changing the destination and interests changes the output
- ☐ The Part 6 chatbot responds conversationally
- ☐ The chatbot correctly routes to different agents based on the question
- ☐ The chatbot remembers previous messages in the conversation

Write-Up Questions

Answer the following questions in a separate document. Each response should be 3–5 sentences unless otherwise specified. Include screenshots where indicated.

Question 1: Agent Architecture
Explain the Supervisor (orchestrator/subagent) pattern used in this lab. What role does the Supervisor play, and how does it differ from the specialist agents? Why is this pattern useful for complex tasks?

≫ *Include a screenshot of your supervisor code (`create_supervisor` call).*

Question 2: Tool Design
Each specialist agent has exactly ONE tool. Why is this a good design choice for a multi-agent system? What might go wrong if you gave all three tools to a single agent instead?

Question 3: Agent Coordination
Look at the Agent Activity Log in the sidebar after running a query.

a) In what order did the Supervisor call the agents?
b) Why does this order make sense for trip planning?
c) What would happen if the Itinerary Agent ran BEFORE the Research Agent?

≫ *Include a screenshot of your Agent Activity Log.*

Question 4: Single-Agent vs. Multi-Agent Comparison
Compare the multi-agent trip plan to the single-agent response.

a) Which response had more specific, actionable information? Why?
b) Which response was better organized? What caused the difference?
c) Identify at least TWO concrete examples where the multi-agent response contained data that the single-agent response lacked.

≫ *Include a screenshot showing both responses.*

Question 5: Trade-offs
Multi-agent systems are not always better. Discuss at least THREE trade-offs or downsides of using a multi-agent approach compared to a single-agent approach. Consider: latency, cost, complexity, and failure modes.

Question 6: Real-World Application
Propose a DIFFERENT real-world scenario (not trip planning) where a multi-agent system would be more effective than a single agent.

a) Describe the scenario and the user's goal.
b) Define at least 3 specialist agents (names, tools, responsibilities).
c) Write a 2–3 sentence supervisor prompt that would orchestrate them.

Question 7: Coordination Protocols
The lab uses the Supervisor pattern. Two alternatives are *peer-to-peer* (agents communicate directly) and *hierarchical* (multiple levels of supervisors). Choose ONE and explain:

a) How it differs from the Supervisor pattern
b) A scenario where it would be MORE appropriate
c) A scenario where it would be LESS appropriate

Question 8: MessagesState & StateGraph

In Part 6 you built a chatbot using `StateGraph` and `MessagesState`.

a) What is `MessagesState` and what role does it play in the graph?

b) Explain the difference between a static edge (`add_edge`) and a conditional edge (`add_conditional_edges`). Give an example of each from your code.

c) Why does the graph need a "synthesizer" node? What would happen if the specialist agents connected directly to `END`?

d) How does `create_supervisor()` relate to the `StateGraph` you built manually? What does it do for you automatically?

≫ *Include a screenshot of the chatbot in action (a multi-turn conversation).*

Submission Checklist

- ☐ `Lab_MA.py` — runs without errors, includes Parts 1–6
- ☐ Screenshots:
  - Supervisor code (Question 1)
  - Agent Activity Log (Question 3)
  - Multi-agent vs. single-agent comparison (Question 4)
  - Chatbot conversation (Question 8)
  - Any additional screenshots supporting your write-up
- ☐ Write-up with answers to all 8 questions